

Per-label gate-boundary thresholds — round 1

Results

The 1,000-trial run's champion was capped by a **bookkeeping detail, not by its model**.

`tune_priority_hash._thresholds_for_trial` selects a single `priority_target_index` for **all 16 priority tags at once**, so the recall target the hardest tag needs — `bank_account_number`, worst-corpus recall 0.60 — is imposed on tags already saturated at worst-corpus recall **1.0000** (`iban`, `itin`, `mrn`, `pin`, `credit_card_number`). Every one of the 16 tags consequently routes to the `source:recall` component, and the shipped champion operates at roughly **0.99 recall against 0.16 implied macro precision**: about four in five of its positive flags are wrong.

Choosing the target **per label** instead — the highest threshold whose bootstrap `ci_lower` still clears 0.90 on every held-in corpus — moves both headline numbers while leaving the gate intact. Nothing was retrained; only `HashCueModel.thresholds` changed.

Arm	macro F2	micro F1	Conclusive gates	Worst recall	p95 (1 core)
champion_1k (incumbent)	0.4835	0.3812	55/55	0.9888	1.697 ms
per_label_v4 (selected)	0.4932	0.4249	55/55	0.9463	1.717 ms

`mp decide`, run against the project's frozen `decision/policy.yaml` over all nine arms, selects `per_label_v4`: it is one of only two arms clearing every hard constraint, and it wins the first preference outright. The decision is recorded at `decision_r1/decision.json`; `decision/` is untouched, so `decision/verify.py` still reproduces the original run's published result over the original eight arms.

micro F1 is where the real movement is: +0.0437, an 11.5% relative gain. macro F2 moves +0.0097 because F2 weights recall 4×, so it barely rewards the precision this change buys.

The two corrections, both found by the gate rather than against it

Three candidates were rejected before one passed. Each rejection identified a defect in the *selection procedure*, which was then fixed on a stated principle — not by tuning against the sealed numbers.

Candidate	Conclusive	Inconclusive	Failures	Defect it exposed
per_label_1k	—	—	2	selection filtered to <code>label_complete</code> corpora, hiding <code>govdocs2</code>
per_label_v2	50/55	5	0	selection ignored that a holdout scope has $\sim\frac{1}{4}$ the statistical power
per_label_v3	53/55	2	0	power scaling added, margin 0.005 too thin for the train→eval recall shift
per_label_v4	55/55	0	0	accepted

First defect — a recall gate is measurable on positive-only gold. The initial selection used only `label_complete` corpora, which excluded `26095_govdocs2-dualjudge` — real government documents, and the held-in twin of the one holdout corpus whose distribution is genuinely different. Its gold carries 12,725 `full_name` and 3,291 address positives, all usable for recall. Blind to them, selection raised `full_name` 4.42× and address 2.26×, and both promptly failed on holdout `govdocs2` (0.8467 and 0.8620). The frozen evaluator scores positive-only corpora for recall exactly as it scores complete ones; the filter was simply wrong.

Second defect — a threshold provable on 3,000 documents is not provable on 150. The splits are 4:1 (5:1 for the two largest), so each holdout scope carries about a quarter of its held-in twin's positives, and a quarter of the statistical power. Selecting on held-in point recall produced thresholds whose `ci_lower` cleared 0.90 on the big held-in corpora and missed it on the small holdout ones — five scopes with support 50–354 landed at `ci_low` 0.861–0.892 while their *point* recall was comfortably above 0.90. Selection now shrinks each held-in scope to its twin's expected support before taking the bootstrap bound. **Only split sizes are used — never holdout labels, predictions, or scores.**

That correction is visible in what it changed: `mrn` 0.94→0.96, `patient_id` 0.96→0.99, `pin/password/military_id` 0.94→0.96, address 0.92→0.94 — four of the five scopes that had been inconclusive, raised without ever consulting the holdout.

Latency

Thresholds-only edits cannot change the compute — same features, same `matmul`, same comparison against a different constant vector — but the 5 ms p95 budget is a hard gate constraint, and an unmeasured constraint is not a passed one. Both models were timed head-to-head at **1 core** on the same deterministic 1,000-document stratified sample:

Arm	p50	p95	p99	docs/s
<code>champion_1k</code>	0.679 ms	1.697 ms	1.866 ms	1007.1
<code>per_label_v4</code>	0.689 ms	1.717 ms	1.893 ms	1007.4

The 20 μs difference is noise. Note these are **not** comparable to the 2.200 ms in `benchmarks/read_depth.json`, which was measured on a different occasion under different machine load; the champion re-measured at 1.697 ms here. The head-to-head pair is the valid comparison.

What did not move, and why

`macro_f2` remains far below the run's original 0.90 aspiration, and this candidate does not change that. A feasibility probe run before this work (`feasibility.json`, verdict **unlikely**) puts the ceiling near **0.589** from the champion's own 95% bootstrap CI upper bound, with the best of five model families over 1,000 trials at 0.4871. The read-depth ladder is flat — 20× more input text moves macro F2 by 0.0037 — so more text is not the missing ingredient either.

The original `budget_exhausted` **route was therefore left in place rather than re-recorded as** `target_infeasible`. That re-recording was planned and is now deliberately deferred: the 0.589 ceiling was derived from arms that *all shared the shared-target-index defect*, so it bounds that threshold policy, not the task. This round moved the number above what several of those arms achieved. The ceiling should be re-estimated before anyone files 0.90 as unreachable.

TL;DR

- The champion was running at ~0.16 macro precision because one shared recall target was applied to all 16 priority tags at once. Fixing that per label is worth **+0.0437 micro F1 (+11.5% relative)** and **+0.0097 macro F2**, with **all 55 priority recall gates still conclusively passing** and latency unchanged at 1.72 ms p95 on one core.
- mp_decide selects per_label_v4 over all nine arms under the project's own frozen policy. Committed as ce1f08b; audit record at audits/t1-c1.json.
- Nothing was retrained. Only HashCueModel.thresholds changed.
- Three candidates were rejected first, each exposing a real defect in the selection procedure: a label_complete filter that hid the one real-document corpus from a recall gate, and a failure to account for holdout scopes carrying ¼ the statistical power of their held-in twins.
- **Still open:** the 0.90 macro F2 aspiration remains out of reach and the ceiling estimate behind it is now stale. Precision is still the binding problem — implied macro precision is roughly 0.19 after this change, up from 0.16. The next lever is the corpus-disagreement question: the same model scores 0.63 on openpii and 0.23 on betterdataai_ner_silver, a 0.40 spread that looks like label-definition disagreement rather than model error, and nobody has measured inter-corpus label agreement.

Model scoreboard

Model	Dataset	F1@ 5	F1@ 3	F1@ 1	Docs/s ec	ms/ doc
current_rules	4000_datax-dualjudge-evalset-1.32k					
current_rules	6589_govdocs2-dualjudge-eval20-3.53k					
current_rules	ai4privacy_pii_masking_eval_10k					
current_rules	betterdataai_ner_silver_eval_10k					
current_rules	nemotron_eval_6k					
current_rules	openpii_pii_eval_38k					
current_rules	pii2_eval_30k					
current_rules	pii_holdout_20k					
hash_sgd	4000_datax-dualjudge-evalset-1.32k					
hash_sgd	6589_govdocs2-dualjudge-eval20-3.53k					
hash_sgd	ai4privacy_pii_masking_eval_10k					
hash_sgd	betterdataai_ner_silver_eval_10k					
hash_sgd	nemotron_eval_6k					
hash_sgd	openpii_pii_eval_38k					
hash_sgd	pii2_eval_30k					
hash_sgd	pii_holdout_20k					

Model	Dataset	F1@5	F1@3	F1@1	Docs/sec	ms/doc
hash_sgd_f2	4000_datax-dualjudge-evalset-1.32k					
hash_sgd_f2	6589_govdocs2-dualjudge-eval20-3.53k					
hash_sgd_f2	ai4privacy_pii_masking_eval_10k					
hash_sgd_f2	betterdataai_ner_silver_eval_10k					
hash_sgd_f2	nemotron_eval_6k					
hash_sgd_f2	openpii_pii_eval_38k					
hash_sgd_f2	pii2_eval_30k					
hash_sgd_f2	pii_holdout_20k					
hybrid_priority_001	4000_datax-dualjudge-evalset-1.32k					
hybrid_priority_001	6589_govdocs2-dualjudge-eval20-3.53k					
hybrid_priority_001	ai4privacy_pii_masking_eval_10k					
hybrid_priority_001	betterdataai_ner_silver_eval_10k					
hybrid_priority_001	nemotron_eval_6k					
hybrid_priority_001	openpii_pii_eval_38k					
hybrid_priority_001	pii2_eval_30k					
hybrid_priority_001	pii_holdout_20k					
tfidf_linear	4000_datax-dualjudge-evalset-1.32k					
tfidf_linear	6589_govdocs2-dualjudge-eval20-3.53k					
tfidf_linear	ai4privacy_pii_masking_eval_10k					
tfidf_linear	betterdataai_ner_silver_eval_10k					
tfidf_linear	nemotron_eval_6k					
tfidf_linear	openpii_pii_eval_38k					
tfidf_linear	pii2_eval_30k					
tfidf_linear	pii_holdout_20k					
embeddingbag_asl	4000_datax-dualjudge-evalset-1.32k					
embeddingbag_asl	6589_govdocs2-dualjudge-eval20-3.53k					
embeddingbag_asl	ai4privacy_pii_masking_eval_10k					
embeddingbag_asl	betterdataai_ner_silver_eval_10k					
embeddingbag_asl	nemotron_eval_6k					

Model	Dataset	F1@5	F1@3	F1@1	Docs/sec	ms/doc
embeddingbag_asl	openpii_pii_eval_38k					
embeddingbag_asl	pii2_eval_30k					
embeddingbag_asl	pii_holdout_20k					
hybrid_priority	4000_datax-dualjudge-evalset-1.32k					
hybrid_priority	6589_govdocs2-dualjudge-eval20-3.53k					
hybrid_priority	ai4privacy_pii_masking_eval_10k					
hybrid_priority	betterdataai_ner_silver_eval_10k					
hybrid_priority	nemotron_eval_6k					
hybrid_priority	openpii_pii_eval_38k					
hybrid_priority	pii2_eval_30k					
hybrid_priority	pii_holdout_20k					
champion_1k	4000_datax-dualjudge-evalset-1.32k					
champion_1k	6589_govdocs2-dualjudge-eval20-3.53k					
champion_1k	ai4privacy_pii_masking_eval_10k					
champion_1k	betterdataai_ner_silver_eval_10k					
champion_1k	nemotron_eval_6k					
champion_1k	openpii_pii_eval_38k					
champion_1k	pii2_eval_30k					
champion_1k	pii_holdout_20k					
champion_1k_bootstrap	8 holdouts (5 complete for F2)				454.5	2.2
hash_sgd_readpass	8 holdouts / fixed 1,000-doc latency sample				984.8	1.0
hash_sgd_readpass	8 holdouts / fixed 1,000-doc latency sample				483.5	2.1
hash_sgd_readpass	8 holdouts / fixed 1,000-doc latency sample				204.6	4.9
hash_sgd_readpass	8 holdouts / fixed 1,000-doc latency sample				146.2	6.8
hybrid_priority_001_readpass	8 holdouts / fixed 1,000-doc latency sample				674.0	1.5
hybrid_priority_001_readpass	8 holdouts / fixed 1,000-doc latency sample				341.6	2.9
hybrid_priority_001_readpass	8 holdouts / fixed 1,000-doc latency sample				175.4	5.7
hybrid_priority_001_readpass	8 holdouts / fixed 1,000-doc latency sample				130.7	7.6
hybrid_priority_readpass	8 holdouts / fixed 1,000-doc latency sample				454.5	2.2

Model	Dataset	F1@ 5	F1@ 3	F1@ 1	Docs/s ec	ms/ doc
hybrid_priority_readpass	8 holdouts / fixed 1,000-doc latency sample				237.5	4.2
hybrid_priority_readpass	8 holdouts / fixed 1,000-doc latency sample				147.8	6.8
hybrid_priority_readpass	8 holdouts / fixed 1,000-doc latency sample				113.3	8.8
perlabel_v4	4000_datax-dualjudge-evalset-1.32k					
perlabel_v4	6589_govdocs2-dualjudge-eval20-3.53k					
perlabel_v4	ai4privacy_pii_masking_eval_10k					
perlabel_v4	betterdataai_ner_silver_eval_10k					
perlabel_v4	nemotron_eval_6k					
perlabel_v4	openpii_pii_eval_38k					
perlabel_v4	pii2_eval_30k					
perlabel_v4	pii_holdout_20k					

Measured 2026-08-25 · n=1,000, n=126,129.